Privacy Enhanced Neural Network using FHE

Antoine Valicon

01/06/2026

Contents

1 Introduction **2**
1.1 Overview . . . 2
1.2 FHE Definition . . . 2

2 Ring Learning With Errors (RLWE) Cryptosystem **2**
2.1 Parameters . . . 2
2.2 Setup . . . 2
2.3 Encryption . . . 3
2.4 Decryption . . . 3

3 Cheon-Kim-Kim-Song (CKKS) FHE Scheme adaptation **3**
3.1 Encoding/Decoding . . . 3
3.2 Encryption/Decryption . . . 4
3.3 Homomorphic operations . . . 4
3.3.1 Addition . . . 4
3.3.2 Multiplication . . . 4
3.4 Bootstrapping . . . 6

4 Application to a Privacy Enhanced Neural Network **6**
4.1 Example usage scenario in real life . . . 6
4.2 Neural Network presentation . . . 7
4.3 Privacy target and adaptation . . . 9
4.4 Results and discussion . . . 10

5 Conclusion **18**
5.1 Room for improvement . . . 18
5.2 Summary . . . 18

1 Introduction

1.1 Overview

Today, most people use Machine Learning and AI in many areas, sometimes without even realizing it. However, to be relevant, Machine Learning needs a lot of data to train and make accurate predictions, classifications, etc... And this can lead to important data privacy questions which we are hearing a lot about especially in Europe. One of the solutions given by Mathematicians and Cryptographers is Fully Homomorphic Encryption (FHE) which we will define in the next sub section. In this report, we will focus on application of FHE in Machine Learning but it can also be applied in other very interessant fields. In order to illustrate our idea, we have successfully implemented a "simple" Privacy Enhanced Neural Network based on MNIST dataset which is able to classify an encrypted digit image.

1.2 FHE Definition

An homomorphism is an application from one set to another which preserves the algebraic structure. In our cryptographic case, the morphism ensures that operations performed on ciphertexts reflect operations on plaintexts. Mathematically, if E is the encryption function, D the decryption function, $\star$ an operation in the plaintext space, and $\circ$ the corresponding operation in the ciphertext space, we have:

$$D(E(m_1) \circ E(m_2)) = m_1 \star m_2$$

There are three kinds of homomorphic encryptions:

- **Partially Homomorphic Encryption (PHE):** It supports only one type of operation (either addition or multiplication) an infinite number of times. Classic examples include RSA (multiplicative) or ElGamal.
- **Leveled Homomorphic Encryption (LHE):** It supports both additions and multiplications, but for a limited and planned number of operations (defined by the parameters of the system).
- **Fully Homomorphic Encryption (FHE):** It allows an infinite number of both additions and multiplications, achieved through a noise-clearing process.

2 Ring Learning With Errors (RLWE) Cryptosystem

2.1 Parameters

The plaintext space is $\mathbb{Z}_t$.
The ciphertext space is the cyclotomic polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. The polynomial degree N should be a power of 2.

2.2 Setup

Key generation : S is a random polynomial (from a uniform distribution) in $\mathbb{Z}_2[X]/(X^N + 1)$, i.e S has ternary coefficients in $\{-1, 0, 1\}$.
We also define a scaling factor $\Delta = \lfloor \frac{q}{t} \rfloor$

2.3 Encryption

We want to encrypt a polynomial $M \in \mathbb{Z}_t[X]/(X^N+1)$. The coefficients of this polynomial represent the plaintext.
First, we generate a random mask $A \in \mathcal{R}_q$ from a uniform distribution.
Then, we generate a random (gaussian distribution) polynomial $E \in \mathcal{R}_q$ with small coefficients, this polynomial represent the noise.
Finally, compute B :

$$B = A \cdot B + \Delta M + E \in \mathcal{R}_q$$

The encrypted couple is : (A, B)

2.4 Decryption

We get the ciphered couple (A, B).
First we eliminate the mask with the key S :

$$B - A \cdot S = \Delta M + E$$

Then we eliminate the noise by rounding to the nearest integer :

$$\left\lceil \frac{\Delta M + E}{\Delta} \right\rfloor \mod t = M$$

Note that M is successfully recovered only if $\left|\frac{E}{\Delta}\right| < \frac{1}{2}$, i.e the coefficients e_i of E satisfy $|e_i| < \frac{\Delta}{2}$.

3 Cheon-Kim-Kim-Song (CKKS) FHE Scheme adaptation

CKKS is a famous FHE scheme relying on RLWE that differs from others by using a different encoding technique in order to encrypt real numbers instead of integers. In the following sections we will present the FHE scheme we implemented which is mainly inspired by CKKS but includes a few differences.

3.1 Encoding/Decoding

We want to encode $\vec{v} = (v_0, v_1, \cdots, v_{\frac{n}{2}-1}) \in \mathbb{C}^{\frac{n}{2}}$ into a polynomial $M(X) \in \mathbb{Z}_t[X]/(X^N+1)$ where t is a modulo coefficient chosen to be smaller than q but large enough to keep all the information.
First, we transform our vector using the complex conjugate as follows :

$$\vec{v}' = \left(v_0, v_1, \cdots, v_{\frac{n}{2}-1}, \overline{v_0}, \overline{v_1}, \cdots, \overline{v_{\frac{n}{2}-1}}\right) \in \hat{\mathbb{C}}$$

The idea is to use an isomorphism between $\mathbb{C}^n$ and $\mathbb{R}[X]/(X^N+1)$. The isomomorphism is given by :

$$\sigma_c : f(X) \in \mathbb{R}[X]/(X^n+1) \longmapsto \left(f(\omega),\, f(\omega^3),\, f(\omega^5),\, \cdots,\, f(\omega^{2n-1})\right) \in \mathbb{C}^n$$

with $\omega = e^{i\pi/n}$ a root of X^n+1. In practice, the isomorphism is a bit modified so it can allow more operations (like rotation) on the vector but it doesn't change a lot from this one. In our implementation we implemented the modified one but we don't use rotation so we could have used this one.
We can represent this isomorphism with matrix multiplications and compute the real vector

$$\vec{m} = \frac{W \cdot J_n \cdot \vec{v}}{n}$$

with J_n the anti identity matrix

and $W = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \omega & \omega^3 & \cdots & \omega^{2n-1} \\ \omega^2 & (\omega^3)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \ddots & \vdots \\ \omega^{n-1} & (\omega^3)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$

Then, we convert $\vec{m}$ into an integer by using a precision factor λ such that $\lfloor \lambda \vec{m} \rceil \approx \lambda \vec{m}$ where $\lfloor \lambda \vec{m} \rceil$ means rounding every component to the nearest integer. The choice of λ must be done carefully because it must not overflow the modulus t during the encoding process but also during any homomorphic operation.
We can finally construct the polynomial

$$M(X) = \sum_{i=0}^{n-1} \lceil \lambda m_i \rfloor X^i \in \mathbb{Z}_q[X]/(X^n + 1)$$

Decoding is very similar, we do the same operations in reverse order, instead of using W we use $W^\top$. We also need to divide by the precision factor at the beginning, but we also need to be careful because it has potentially increased during multiplications.
Concerning our implementation, we distinguished precision factor λ and scaling factor Δ (which we will introduce in the next section) but in CKKS they seem to be the same which leads to be careful with it at decoding in our case.

3.2 Encryption/Decryption

The encryption/decryption procedure is the same as described in section 2. (However, the mask A is multiplied by -1 in B)
The only difference is the computation of Δ, since it was not very clear how Δ was computed in CKKS Scheme we arbitrarily choose it to be a power of 2.
Moreover, q will be denoted q_0 (we call it size MODULO_0_BIT_SIZE in our program) and will also be a big power of 2. We are going to explain why this change in the next section concerning operations.

3.3 Homomorphic operations

3.3.1 Addition

We can perform addition between a ciphertext and a plaintext or between two ciphertexts.
Recall that the ciphertext is a couple (A, B) with $B = A \cdot S + \Delta M + E \in \mathcal{R}_q$. The ciphertext/ciphertext addition is very simple :

$$(A_1, B_1) + (A_2, B_2) = (A_1 + A_2, B_1 + B_2)$$

Note that both ciphertext need to be in the same space $\mathcal{R}_q$.
The ciphertext/plaintext addition is similar but the plaintext must be an encoded polynomial P scaled with Δ. It gives :

$$(A_1, B_1) + \Delta P = (A_1, B_1 + \Delta P)$$

We can verify that we recover the correct sum after decryption. The noise after ciphertext/ciphertext addition would be $E_1 + E_2$ and if Δ is chosen large enough this should never overflow $\frac{\Delta}{2}$

3.3.2 Multiplication

Same as before we can perform multiplication between a ciphertext and a plaintext or between two ciphertexts.
The only difference here is that each multiplication is going to consume a multiplicative level. This is why

we noted the modulus q_0 in section 3.2, q_0 is the base modulus but the first operations are going to happen modulus $q_L = q_0 \times \Delta^L$ where L is the max multiplicative level. After one multiplication, we decrease q_L by one level and we get $q_{L-1} = q_0 \times \Delta^{L-1}$, in order to do this and keep track of our correct message we perform an operation called rescaling.

Ciphertext/Plaintext multiplication :

As in homomorphic addition, the plaintext must be an encoded polynomial P scaled with Δ. We have :

$$(A, B) \times \Delta P = (A \times \Delta P, B \times \Delta P)$$

The problem now is the Δ appearing in front of A and B. We can't simply divide by Δ since we are $\mod q_l$ where $l > 0$ is the current multplicative level. In this case we also need to divide q_l by Δ, this is the **rescaling** operation.

To be convinced that it works, we just need to see the modulus notation as an equation. For example, $\Delta B \equiv \Delta(-A \cdot S + \Delta M + E) \mod Q$ is equivalent to

$$Qk + \Delta(A \cdot S + \Delta M + E) = \Delta B$$

for some integer k, then we can divide by an integer Δ and get the new modulus notation :

$$\frac{\Delta B}{\Delta} \equiv \frac{\Delta(-A \cdot S + \Delta M + E)}{\Delta} \mod \frac{Q}{\Delta}$$

i.e $B \equiv -A \cdot S + \Delta M + E \mod \frac{Q}{\Delta}$.

Ciphertext/Ciphertext multiplication :

A naive multiplication would give us :

$$(A_1, B_1) \times (A_2, B_2) = (A_1 \times A_2, A_1 B_2 + A_2 B_1, B_1 \times B_2)$$

which represents a ciphertext for $\Delta^2 M_1 M_2$ but this is not a couple so our goal is to get :

$$(A_1, B_1) \times (A_2, B_2) = (A_1 \times A_2, B_1 \times B_2)$$

This operation is called **relineratization**.
Let

$$D_0 = B_1 \times B_2$$
$$D_1 = A_1 B_2 + A_2 B_1$$
$$D_2 = A_1 \times A_2$$

A formula for multiplication is :

$$(A_1, B_1) \times (A_2, B_2) = (D_1, D_0) + \left\lceil \frac{D_2 \cdot evk_g}{g} \right\rfloor$$

where $evk_g = (A', -A' \cdot S + E' + gS^2)$ with A' a new random mask and E' a new random error polynomial in $\mathcal{R}_q$ and g is a large integer power of 2 below q_L^2.
evk_g is actually a ciphertext of gS^2 using the key S without any scaling factor Δ, evk_g is called an evaluation key. (A demonstration why it works might be given in an another version of this document later...).
At this point we have a formula for the ciphertext of $\Delta^2 M_1 M_2$ but we also need to rescale it as in the Ciphertext/Plaintext mutiplication, the **rescaling** operation is the same.

3.4 Bootstrapping

As previously shown, our scheme is leveled, which means that each time we perform a multiplication and its subsequent rescaling, the global modulus size lowers. When we reach the end of the allowed operations, we are left with a ciphertext where the noise has grown too large compared to the remaining modulus space, making any further computation impossible.
In order to perform more homomorphic operations on ciphertext, we need to reset the multiplicative level, this procedure is called **bootstrapping**.
We did not implement this procedure in our project due to its big cost and complexity. While bootstrapping successfully lowers the noise to a manageable level and resets the parameters, it requires a massive amount of computation and is very long to execute. We avoid implementing it because we don't need to perform a lot of multiplications so we adapted our implementation by using big parameters.

4 Application to a Privacy Enhanced Neural Network

4.1 Example usage scenario in real life

We can easily imagine a scenario where FHE is useful, for example for hand written digit recognition :
Imagine that, for some reason, the only way to access your mail account is by hand writing every digit of your code on your computer.
In order to recognize the digits you gave, the mail server asks a third-party server which hosts a digit recognition algorithm, of course it's a really bad practise and in this example the mail server could implement the algorithm on their server but for more complex tasks (like face recognition or LLM) they don't always have access to the specific algorithms.

Obviously, neither you nor the mail server wants this third-party server to know these digits and this is where the FHE arrives ! The third-party server can implement the digit recognition algorithm which supports an FHE scheme. It implies that the mail server must also use FHE to encrypt and decrypt the value. As a result, the third-party never knows your password but is able to give the mail server a valid recognition value, it looks kind of magical [1].
Since passwords are sensitive data, the previous example is not very realistic, it's only to illustrate our implementation in the next section. However, it could be a useful protocol concerning general data privacy as mentioned in the introduction, such as face recognition, etc...

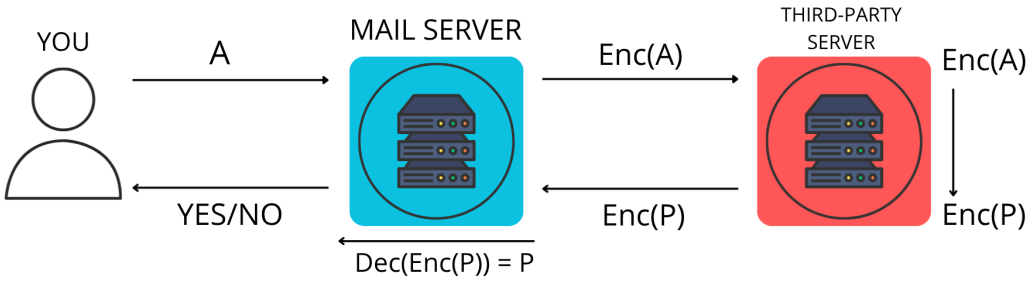


Figure 1: Example usage scenario

4.2 Neural Network presentation

Hand-written digit recognition using MNIST is the basic neural network that we learn when we start deep learning. It consists of two datasets, one for training our network and one for testing.
Every image are 28×28 pixels representing a hand-written digit between 0 and 9. An image is represented by a vector of size 784, in the training set we have approximately 60000 images with their label and 10000 in the testing data set. In addition, the images are all shuffled so the model doesn't learn any bias.
Neural network consists of two main actions : **Inference** and **Training**, inference can be summarized as a prediction while training is an action done at the beginning in order to find the optimal parameters.

At the beginning we initialize randomly the matrix and vectors W_1, b_1, W_2, b_2 (parameters values representing weights and biases), initialization can vary but in our case we chose uniform distribution between $[-0.5, 0.5]$, there are also other techniques such as dividing by the variance which can influence the convergence rate during training, but we wanted to do as simple as possible and the results were correct.
Then, the training images "feed" the network with a feed forward function which is basically matrix multiplications and represented in the following diagram. [2]

Our model consists of 3 layers, with one input layer (in red), one hidden layer (in yellow) with 64 neurons (number of neurons is a parameter we can also vary to adjust the model and computation time) and one output layer (in green). The output layer represents a probability vector, for example the component p_3 is the probability that the input image is a 3.

As you can see there are also functions f and g, they represent activation functions and they are used to introduce non-linearity in the model, some famous functions are **ReLU** (rectified linear unit) which is $\max(0, x)$, **Sigmoid** which is $\tanh(x)$, **Softmax** which is $\frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$, and many other... Before doing any FHE with the model, f was the ReLU function and g was the Softmax function but we will come back later to this point.

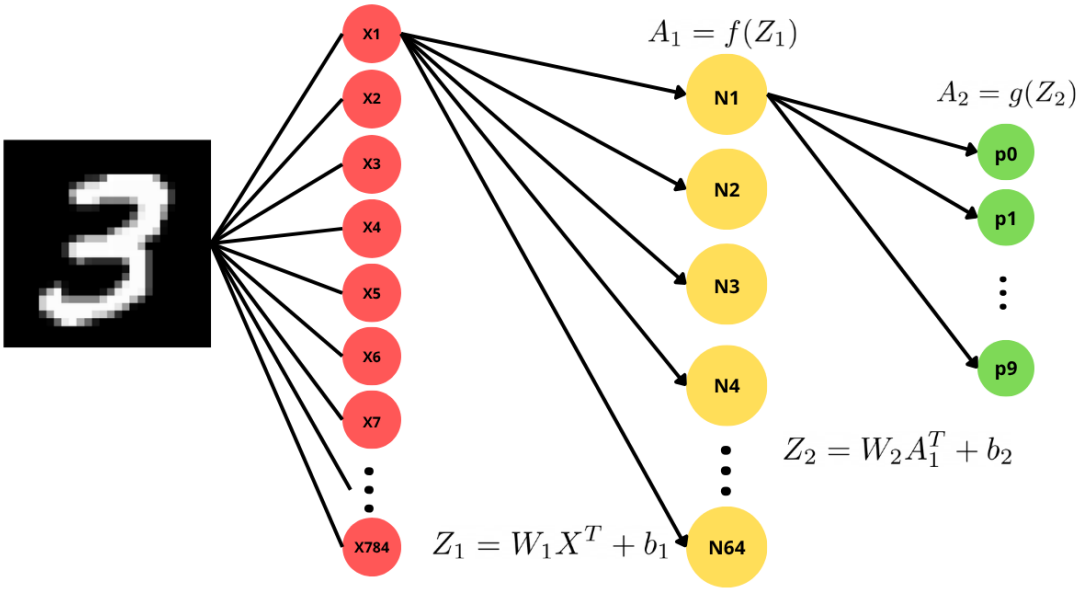


Figure 2: Neural network representation (precisely the feed forward function used for prediction)

After the feed forward function [2] the output vector represents probabilities but the model doesn't know yet if they are correct or not, this is where the training action is performed with a back propagation algorithm. In our case we made a basic stochastic gradient descent algorithm, i.e we subtract the feed-forward output with the "perfect" prediction vector (so a vector looking like $(0, 0, 1, 0, ..., 0)$), which highlights with a negative value where the error is in our the feed forward prediction. The new subtracted vector is dZ_2, we inject it from the right in Fig [2] and it goes from right to left by computing dW_1, db_1, dW_2, db_2 representing the gradients, the new functions used are the derivative of f and g so for ReLU it's $\mathbf{1}_{x \geq 0}$ and for softmax it's more complicated but it's actually done by subtracting the vector before.
At the end we have all the gradients dW_1, db_1, dW_2, db_2 and we can update the W_1, b_1, W_2, b_2 using the gradients and a coefficient parameter called learning rate α. Training is just repeating this for every image.

In our implementation we also made another small optimization for computing and for training by using mini-batch, it doesn't change the main idea but instead of working directly on the 60 000 training images, we make some iterations called epoch where we pick small random batches in the training set and we train the model using those batches. The batch size is another parameter we can choose to get the best accuracy, and it's very useful because it reduces computation (can be parallelized with GPU) without reducing the model accuracy and convergence rate. After training, we can measure accuracy and cross-entropy loss using the test dataset to see if our model is efficient or not.

4.3 Privacy target and adaptation

After understanding how a neural network works we saw two different ways to add FHE to it :

- The first way is the most privacy-friendly but also the most ambitious : we encrypt all the dataset and train the model using the encrypted dataset. As a result, we have a trained model with encrypted parameters. In fact, because of the update parameters operation in the back propagation algorithm, it's impossible to obtain plain parameters using FHE with this method. So we can't use the trained model for prediction unless we have the cipher key. In a symmetric-key crypto system it doesn't make really sense, because the model would need to be trained for every person with a key and training is very computationally expensive. It makes more sense in public-key cryptography where the data is enciphered with one public key and a secret key is securely shared between some people.
- The second way is to train a model using a plain dataset, so we can obtain plain parameters and then make predictions on ciphered data, i.e the example scenario. In this way, only the feed forward function needs to be adjusted to use FHE, it's simpler, less computationally demanding and keeps your input data private. We can also use it with public-key or symmetric-key cryptography.

We have implemented the second way using symmetric cryptography, the procedure to make a prediction on an encrypted image is as follows :
1 - Train the model and generate a parameters file using plain training dataset
2 - Encode an image in a correct way to use FHE properties in RLWE
3 - Generate a RLWE key
4 - Encrypt the encoded image
5 - Feed the FHE feed forward function with the encrypted image
6 - Get encrypted vector result and decrypt it with the key
7 - Decode the decrypted vector result
8 - Make a prediction.

We will explain some details and arbitrary choices we made for each of these steps:

1. We had to adapt the model a little bit to be FHE friendly, since our FHE implementation can only perform multiplications and additions, it's hard to approximate non linear functions like ReLU, so we replaced ReLU with a Square function $f(x) = x^2$. Therefore we had to adapt some other parameters like the learning rate to obtain correct results. Concerning Softmax we didn't replace it, but we will explain in step 5 and 8 how we manage it.

2. Here we made another arbitrary choice by encoding every 784 components of the image instead of the whole vector image. It's clearly not the most optimized way to do it since we could also encode the whole vector into one polynomial as we explained 3.1.
   We made this choice to also make things simpler because it's difficult to perform dot products between matrices when the whole vector is encoded, it's possible with different homomorphic operations like rotate, but we only implemented the basic ones. To make it clear, if M is an encoded vector, in our way of encoding : $M[i]$ is the i-th encoded component of M, but in the most optimized way $M[i]$ doesn't make sense because M is a polynomial. It could make sense, depending on the encoding method but anyway after encryption it doesn't. Therefore we realized that our encoding method was a bit overkill for what we wanted to do.

3. A user of the program is not really aware that he is using a key because a polynomial as a key is not a very human way to store a key especially if the polynomial degree is high.

4. The Encrypt function is the same as in 3.2 To generate the noise polynomial, we use a gaussian distribution and to simulate this distribution from uniform distribution function in C we used the Box-Muller transformation :
   Suppose U_1 and U_2 are independent samples chosen from the uniform distribution on the interval $[0, 1]$.Then,
   $$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$
   follows a standard normal distribution.
   In addition, since FHE needs huge numbers to work correctly because of the scaling factor the standard C type for integers were not enough to store these values so we use the **GNU MP** library and the **mpz_t** type which can handle integers as big as our computer memory capacity.

5. We adjusted the plain feed forward algorithm (with x^2 as activation function) with FHE operations, however we didn't implement the last step with Softmax, therefore we obtain the encrypted vector Z_2 at the end of FHE feed forward. There is only on cipher to cipher multiplication operation because of the activation functions, the other operations are addition and multiplication with encoded plaintext parameters.

6. Decrypting is the same as 3.2 decryption, we decrypt every value of the vector one by one using the modulo that has been decreased when performing Homomorphic multiplication.

7. Decoding is the same as 3.1 but adapted to our structure. As mentioned, we had to keep track of the precision factor because it increases with every multiplication, so we don't divide by the same precision factor we used to encode.

8. At this step we get plain Z_2 and not A_2 like at the end of the plain feed forward function but it's not a big problem, at least for prediction. Recall that $A_2 = \mathbf{softmax}(Z2)$, but **softmax** is mainly used for backpropagation and in our case we just want a prediction (no back propagation and training). What **softmax** does is transforming Z_2 vector components into a probability distributed vector, therefore the biggest probability in A_2 will be the biggest number in Z_2, thus to do the prediction we just take the index of the maximum value in Z_2. It simplified things, and we didn't have to implement a homomorphic softmax.

4.4 Results and discussion

We will now present some results we obtained, we will interpret them at the end of this section.
As we mentioned earlier we trained the model for FHE with the square activation function instead of a classic ReLU, here is a comparison of accuracy and cross entropy loss between the two models during training :

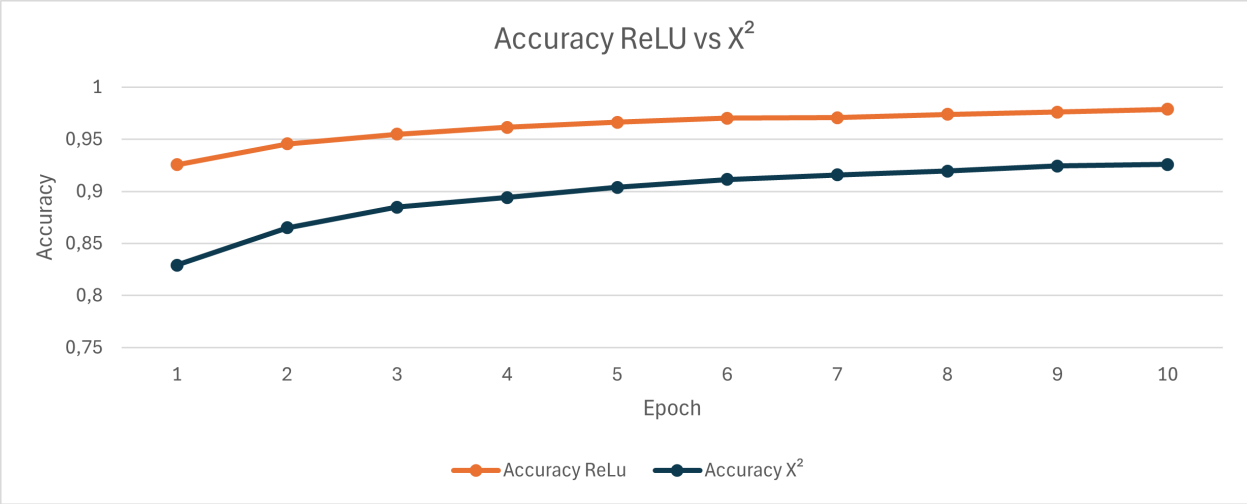


Figure 3: Training accuracy evaluation with ReLU vs x^2 as activation function f

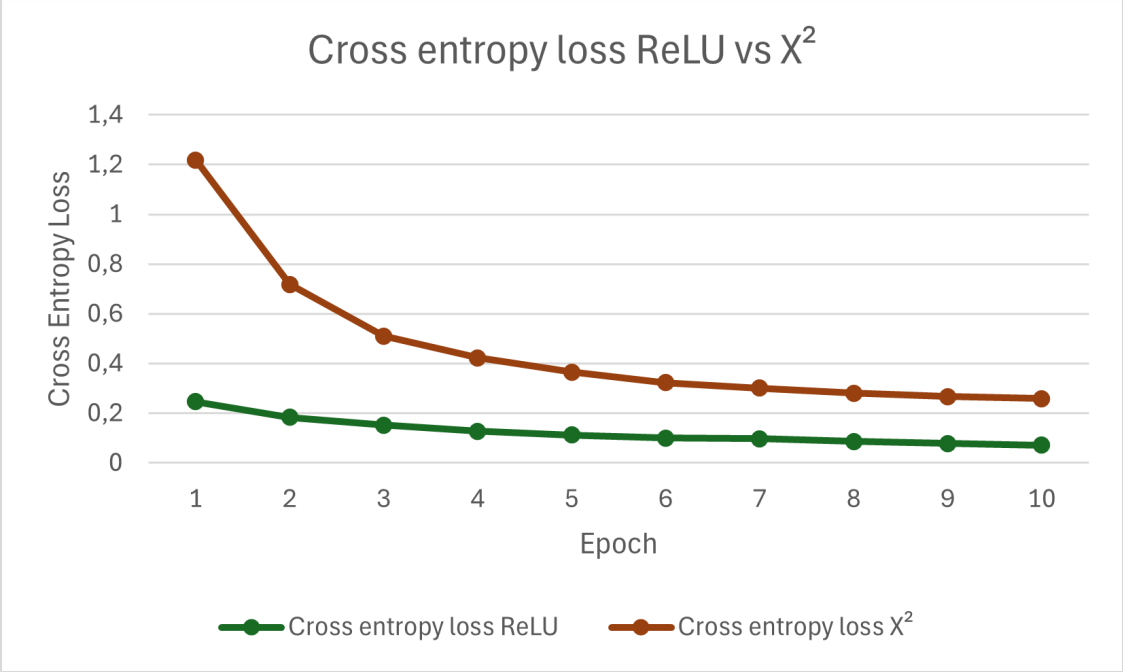


Figure 4: Cross Entropy loss evaluation with ReLU vs x^2 as activation function f

Neural network's goal is to minimize Cross entropy loss and maximize accuracy, as we can see the ReLU function gives better results on the trained data set but x^2 is also fine with around 92% accuracy. We tested the x^2 model on the test dataset and we also got an accuracy of 92% (i.e no overfitting).

We can summarize our neural network hyperparameters in the following table :

Hyperparameter	Value
Learning rate α	0.01
Input layer number of features	784
Hidden layer number of neurons	64
Output layer number of classes	10
Batch size	128

Table 1: Neural network training hyperparameters summary

Concerning the FHE ciphering and encoding, we started by using these parameters:

Parameter	Value
Precision factor λ	2048
Polynomial degree N	4
q_0	2^{100}
Scaling factor Δ	2^{20}
Multiplicative level L	3

Table 2: FHE first parameters used

Since our feed forward function is doing only 3 multiplications (multiplicative depth) for each cipher, the multiplicative level will always be 3.
Recall the parameters choice in 3.4, we need to set a q_0 larger than the plaintext modulo t, the encryption will be done modulo $q_L = q_0 \times \Delta^L = 2^{140}$ in our case and this modulo will decrease at each multiplication level.
We took the parameters in Table 4 randomly but we were already able to perform predictions. In addition, we also noticed the limits of our implementation concerning implementation time :

Number of predictions	Time for plaintext (s)	Time for ciphertext (s)
1	0.000128	0.171783
10	0.00061	1.898766
20	0.001284	3.34412
30	0.001863	5.032443
40	0.002397	6.657875
50	0.002936	8.458753
1000	0.7	166.2

Table 3: Predictions time for plaintext data and ciphered data

Concerning accuracy on the 1000 predictions we got 82% with FHE predictions.
We also evaluated polynomial degree influence on computation time :

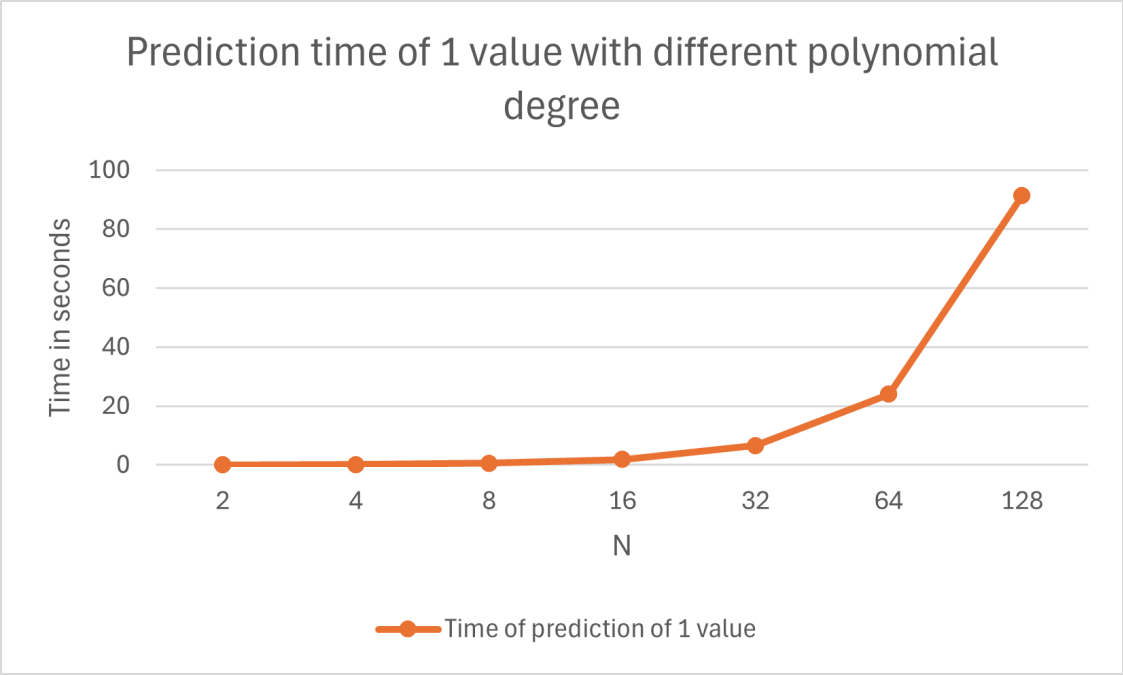


Figure 5: $(X^N + 1)$ degree influence on computation time

For the next tests we kept the polynomial degree to $N = 4$, otherwise it was too long.
We measured accuracy and computation by varying some parameters :

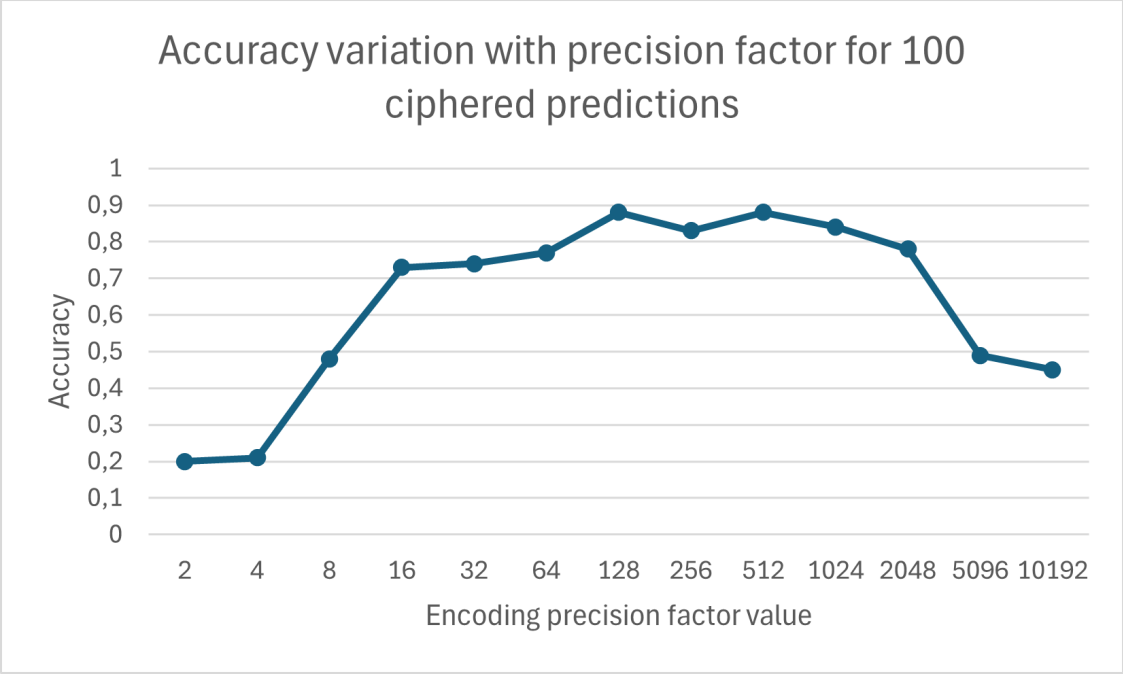


Figure 6: Accuracy variations with different precision factor λ

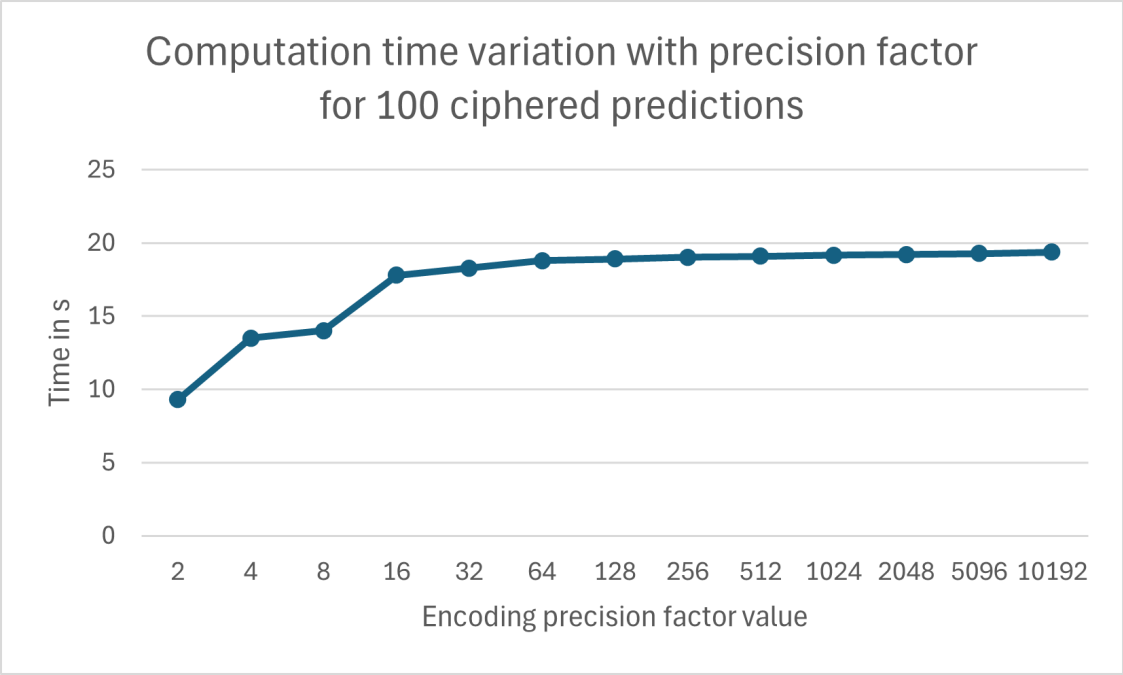


Figure 7: Computation time variations with different precision factor λ

Therefore we kept $\lambda = 512$ for the next tests.

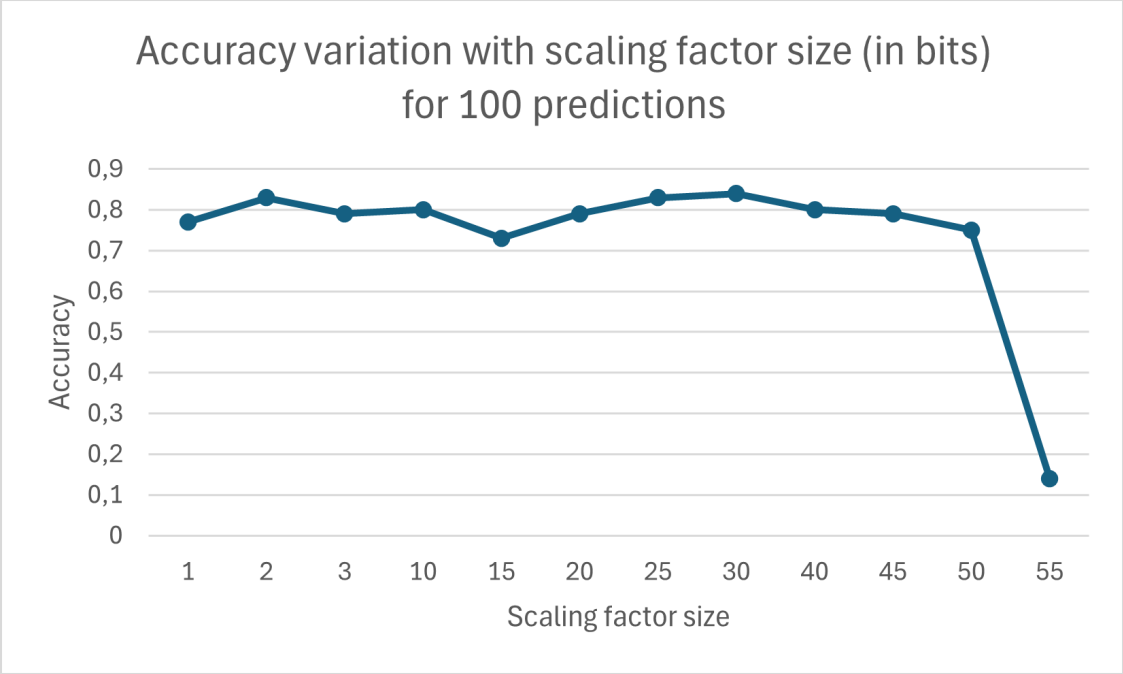


Figure 8: Accuracy variations with different scaling factor Δ

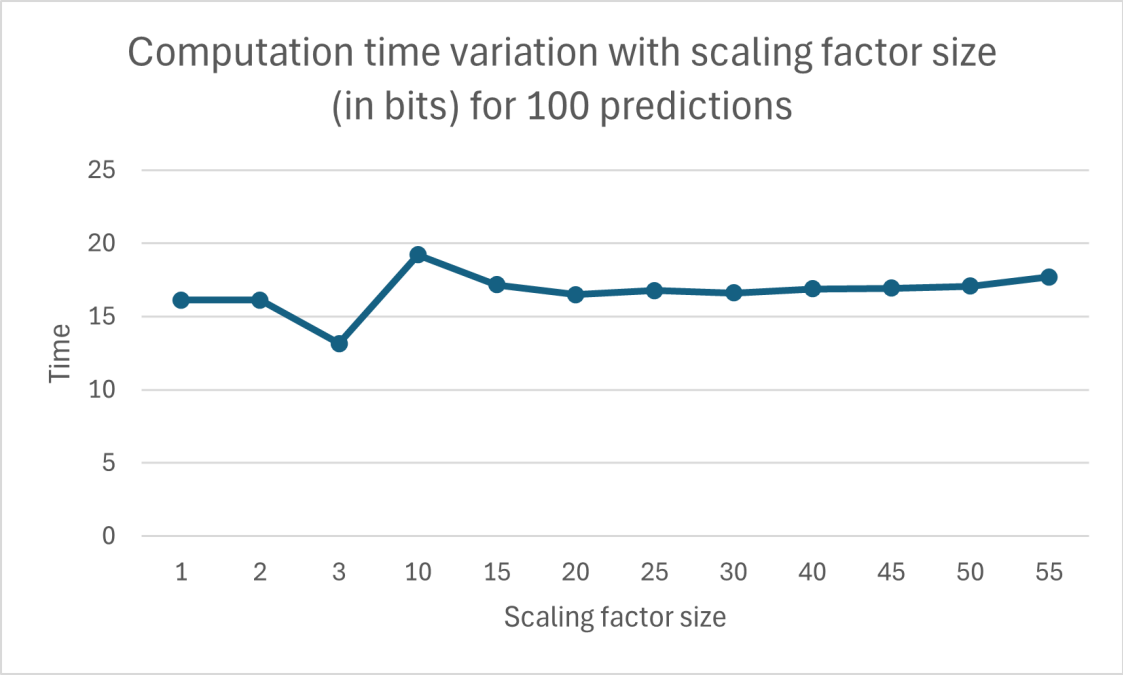


Figure 9: Computation time variations with different scaling factor Δ

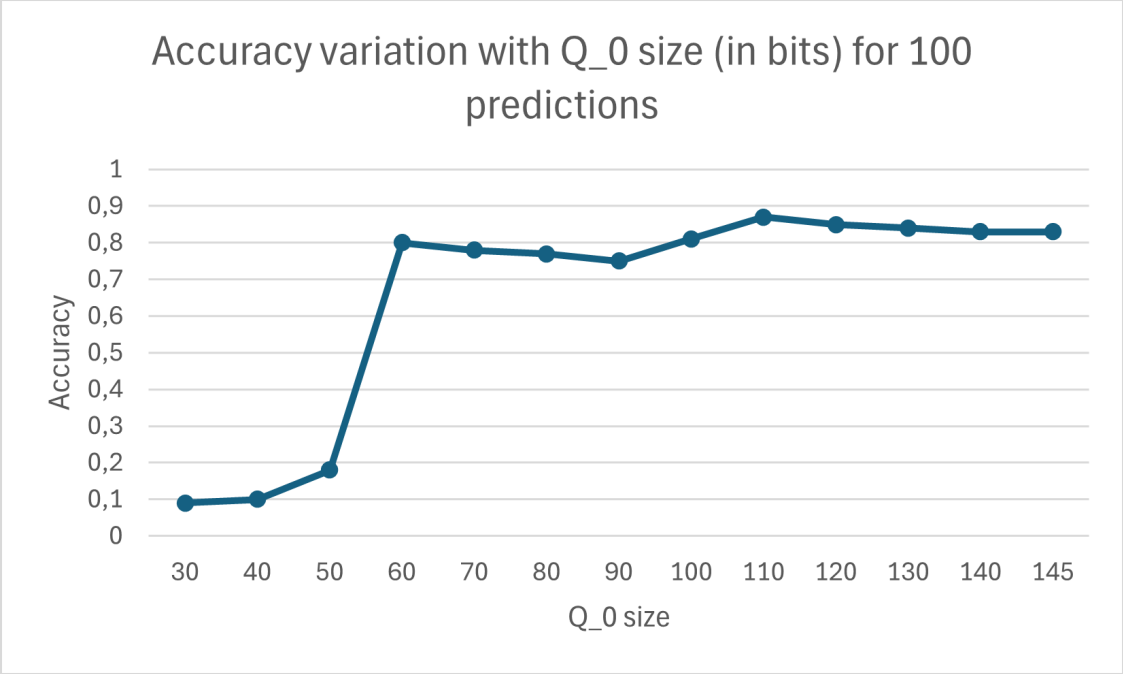


Figure 10: Accuracy variations with different q_0

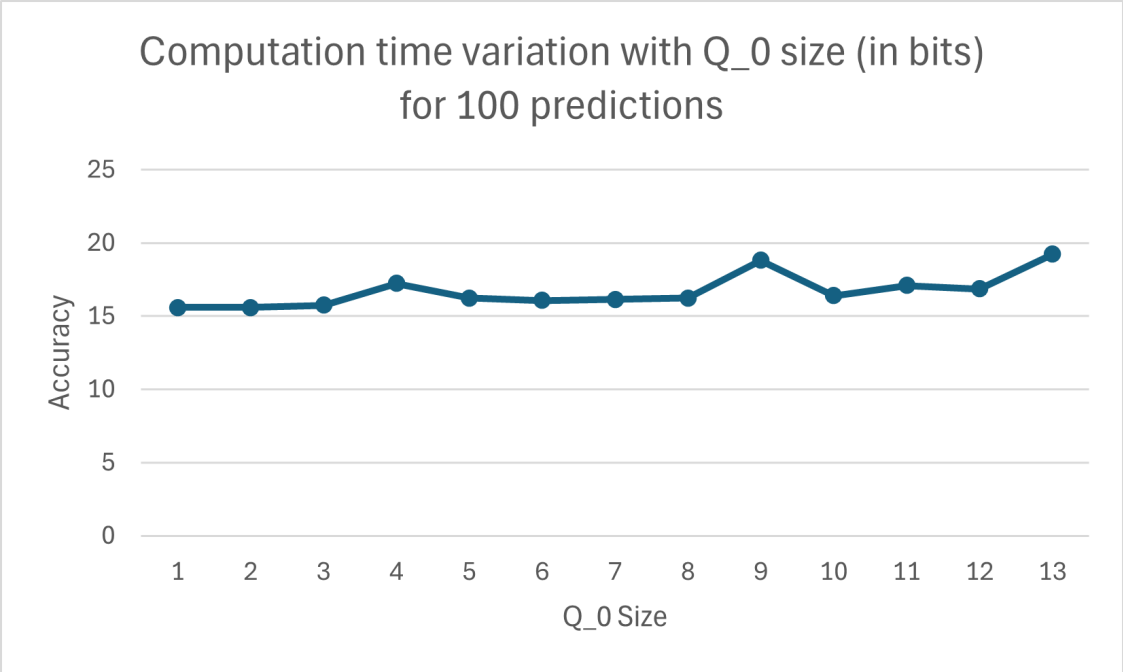


Figure 11: Computation time variations with different q_0

The final chosen parameters are :

Parameter	Value
Precision factor λ	512
Polynomial degree N	4
q_0	2^{100}
Scaling factor Δ	2^{10}
Multiplicative level L	3

Table 4: FHE first parameters used

There are certainly some errors in measuring because we were able to measure only on 100 random predictions and it was too long to do it on more predictions every time. It's also not the best way to evaluate parameters, because some are linked with others

At the end the best accuracy we got was around 90% on 1000 predictions (it differs from the figures because we corrected a small bug atfer). There are many factors that could explain this little loss in accuracy :

- Firstly, by definition of RLWE there is an error in the cryptosystem that needs to be erased at the end during decrypting, but this error grows on every operation and especially on multiplication. To be successfully deleted the error needs to be lower than $|\frac{\Delta}{2}|$ during decrypting, i.e the Δ needs to be as big as necessary however according to our results we saw that increasing size of scaling factor wasn't increasing the accuracy, and it's even worse after size of 50. The decreasing of accuracy after size 50 can be explained because of multiplication operation, since we do $Enc(\Delta M_1 \times \Delta M_2)$ then Δ becomes Δ^2 , i.e his size becomes 100 bits which is larger than the modulo size and we lose the message M in a modulo wrap before rescaling it. However the low Δ having the same accuracy as bigger Δ is trickier to explain, either we made a mistake in the implementation, or the size of Δ isn't actually an issue and the error is never too big to be erased or insignificant to make predictions.

- Another source of error could be the float rounding made during encoding, and this error increases every time we make multiplications between encoded values.
- The most likely source of error, and we saw it with the precision factor measurement, is the modulo of the plain text space t. In our project, we haven't really defined this value other than based on the size of data type used which is a bad practise, therefore the encoded values MUST NOT exceed the size of a **int64_t** so they must lie between $[-2^{63}, 2^{63}]$, if the encoded data exceeds this interval (during encoding or any operations) it's an undefined behavior and we can't decode the value. This is why increasing the precision factor too much is problematic. (We made the encoding at the beginning before using **mpz_t** big integer type so we haven't changed it yet).

Concerning q_0, it doesn't seem to affect the accuracy a lot, however it must not be smaller than the plain text space (as we saw in the results) otherwise same as before there will be a modulo wrap during an operation and we couldn't recover the encoded value at the end. Our hypothesis is that there is a small mix of every type of error we just described (and maybe some we don't even know) which at the end makes wrong predictions especially for more difficult numbers to recognize (like a 7 looking like a 1)...

In terms of computation time, the polynomial degree has the biggest influence on this and sadly this is also a big factor of security... therefore with $N = 4$ our implementation is not secured at all and we could easily recover the key with bruteforce attack.
Here also, a lot of factors play a role in this computation time :

- Firstly, we implemented the naive polynomial multiplication algorithm in $O(n^2)$ whereas FHE libraries use algorithms like NTT (RLWE variant of DFT) in $O(n \log(n)$.
- Also our way of encoding messages is a really bad way of encoding in terms of complexity and also security.
  Complexity : Instead of representing our vector by one polynomial we represent it by 784 polynomials... so we will do more polynomial multiplications.
  Security : It's not semantically secured since we can guess the size of the image without the key, but it's not very important in this case because an attacker may have access to the FHE feed forward function and knows that it's a 28×28 digit recognition algorithm.
- Finally, an important aspect is our coding skills in C. Indeed, at the end, structures became very complicated to understand, to allocate, to free... so we've done the things how we understood it without worrying about optimization.

The other parameters don't have a huge computation time impact because most of them are handled by the GNU MP library which is very optimized at low level to deal with big numbers.

On the security side, as we said our implementation is bad however it's not trivial to estimate an accurate security level of a RLWE implementation because it depends on a lot of parameters. For example, the documentation of the Microsoft SEAL library (FHE library of Microsoft) gives the following table of parameters and security level.

n	Bit-length of default q	
	128-bit security	192-bit security
1024	29	20
2048	56	39
4096	110	77
8192	219	153
16384	441	300
32768	885	600

Figure 12: Pairs of N and q_0 with their security level (from SEAL manual)

Their implementation is different but is also based on the RLWE cryptosystem, and we can see that we are far from their polynomial degree value. Currently, there is no NIST standard for homomorphic encryption (FHE) schemes, unlike AES or RSA, but this is an ongoing process...

5 Conclusion

5.1 Room for improvement

The first improvement we should do is correcting any conceptual bug in the code in order to fix the accuracy problem like the plaintext modulo t implementation we evoked in the discussion 4.4. Once we are sure the code is perfectly correct, we should enhance is security aspect, i.e increasing the size of N (the polynomial degree), but we saw that it was a performance killer so we must also optimize the code.
To do this we must improve algorithms like polynomials multiplication by implementing NTT (a RLWE variant of DFT) for example.
We must also clean and upgrade our code to avoid any useless computation (like computing the big parameters every time or the big W matrix in the encoding section).
With fixed parameters we could also allocate every memory needed at the beginning which would drastically improve performance.
When we have reached a good computation time with a good security level we could think of improving our model using better approximation of ReLU using polynomial approximation.
Finally, we could implement the FHE training part of the neural network on ciphered dataset. It involves implementing a true FHE scheme with bootstrapping which would be a very hard task to do.

5.2 Summary

To conclude, we successfully implemented a neural network and adapted it to FHE cryptosystem based on CKKS in order to perform ciphered digit recognition using the MNIST dataset. This project served as an introduction to lattice-based cryptography, focusing on the RLWE cryptosystem and the fascinating world of FHE. Our final implementation is far from perfect, with issues regarding computation time, accuracy and security but it gives us a big room for improvement.

References

[1] R. Ko, *The Beginner's Textbook for Fully Homomorphic Encryption*.

[2] C. Laine, K. Laine, et R. Player, *SEAL Manual*, Microsoft Research, 2017. `https://www.microsoft.com/en-us/research/wp-content/uploads/2017/12/sealmanual.pdf`.

[3] J. H. Cheon, A. Kim, M. Kim, et Y. Song, *Homomorphic Encryption for Arithmetic of Approximate Numbers* (CKKS original paper), IACR Cryptology ePrint Archive, 2016. Disponible sur : `https://eprint.iacr.org/2016/421.pdf`.